

الكلية متعددة التخصصات - ورازات
+oYXLol+ +oX+XKHXL+ - UoOЖoЖo+
FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE



Rapport Conception

Prepared By:

**Alkor Salma
Younes Stifa**

Presented To:

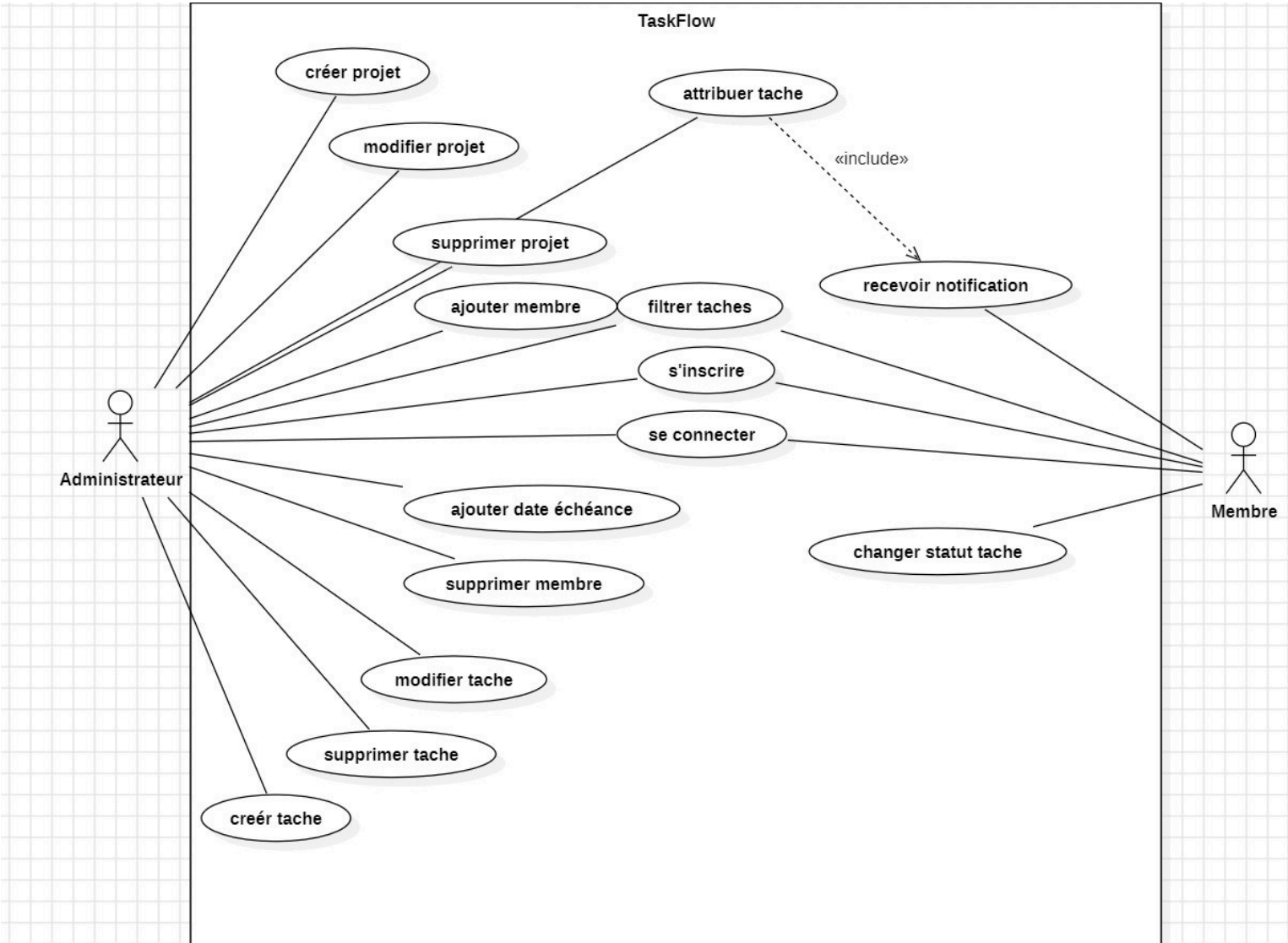
PROF. Ismail Labihi

Introduction :

Le projet **TaskFlow** est une application de gestion de tâches collaboratives qui nous permet de travailler ensemble au sein d'un même projet. Cette application offre différentes fonctionnalités telles que la gestion des utilisateurs, la création des projets, la gestion des tâches, les notifications ainsi que le filtrage des tâches selon plusieurs critères.

L'objectif principal de ce mini-projet est d'appliquer les concepts de la conception logicielle à travers la modélisation UML et l'utilisation des design patterns. Pour cela, plusieurs diagrammes UML ont été réalisés afin de représenter les différents aspects du système : les fonctionnalités, la structure statique ainsi que le comportement dynamique de l'application et aussi d'utiliser la documentation .

Diagramme de cas d'utilisation :



Le diagramme de cas d'utilisation représente les différentes fonctionnalités offertes par l'application ainsi que les interactions entre les acteurs et le système.

Dans notre système, deux acteurs principaux ont été identifiés :

- L'Administrateur
- Le Membre

L'administrateur possède des privilèges avancés lui permettant de gérer les projets et les tâches. Il peut créer, modifier ou supprimer un projet, ajouter des membres, créer des tâches et attribuer ces tâches aux utilisateurs.

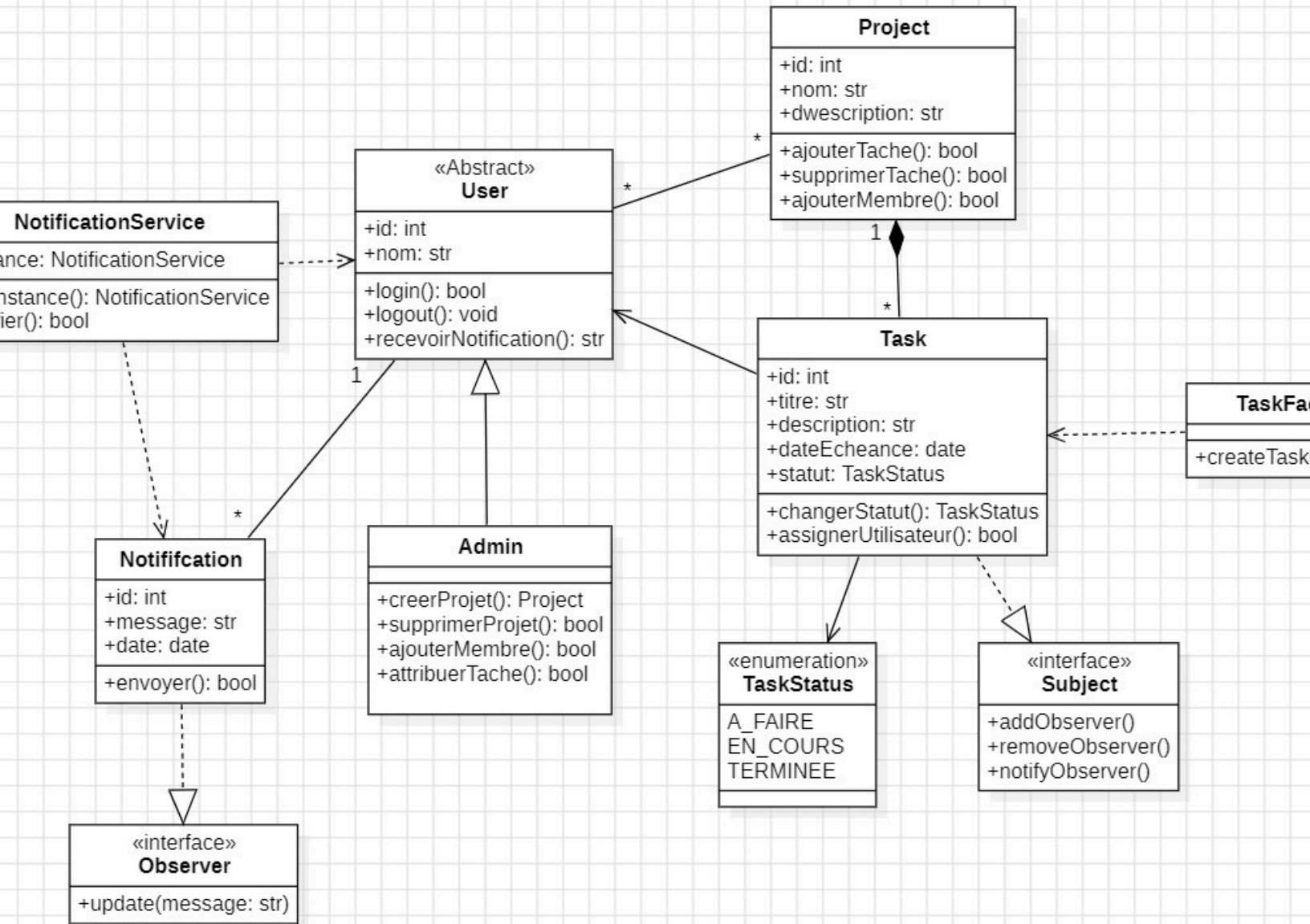
Le membre représente un utilisateur simple du système. Il peut consulter les tâches qui lui sont attribuées, modifier le statut d'une tâche et recevoir des notifications.

Le diagramme montre également les principales fonctionnalités du système telles que :

- l'inscription,
- la connexion,
- la gestion des projets,
- la gestion des tâches,
- la réception des notifications,
- et le filtrage des tâches.

Une relation <<include>> a été utilisée entre le cas d'utilisation « Attribuer tâche » et « Recevoir notification ». Cette relation signifie que lorsqu'une tâche est attribuée à un utilisateur, le système exécute automatiquement le processus de notification.

Diagramme de Classe :



Le diagramme de classes représente la structure statique de l'application. Il décrit les différentes classes du système, leurs attributs, leurs méthodes ainsi que les relations entre elles.

Plusieurs classes principales ont été identifiées :

- User
- Admin
- Project
- Task
- Notification
- NotificationService
- TaskFactory

La classe User représente les utilisateurs du système. Elle contient les informations principales d'un utilisateur telles que l'identifiant, le nom, l'email et le mot de passe.

La classe Admin hérite de la classe User. Cette relation d'héritage montre qu'un administrateur est un utilisateur possédant des fonctionnalités supplémentaires comme la gestion des projets et des membres.

La classe Project représente les projets collaboratifs. Un projet peut contenir plusieurs tâches et plusieurs membres.

La classe Task représente les tâches du projet. Chaque tâche possède un titre, une description, un statut et une date d'échéance. Une tâche peut être attribuée à un utilisateur.

La classe Notification permet de gérer les messages envoyés aux utilisateurs lorsqu'une action importante est effectuée dans le système.

Le diagramme contient également plusieurs relations importantes :

- une relation de composition entre Project et Task,
- une relation d'association entre Task et User,
- et une relation d'héritage entre Admin et User.

Plusieurs design patterns ont été intégrés dans cette conception :

Singleton

Le pattern Singleton a été utilisé dans la classe NotificationService afin de garantir l'existence d'une seule instance du service de notification dans toute l'application.

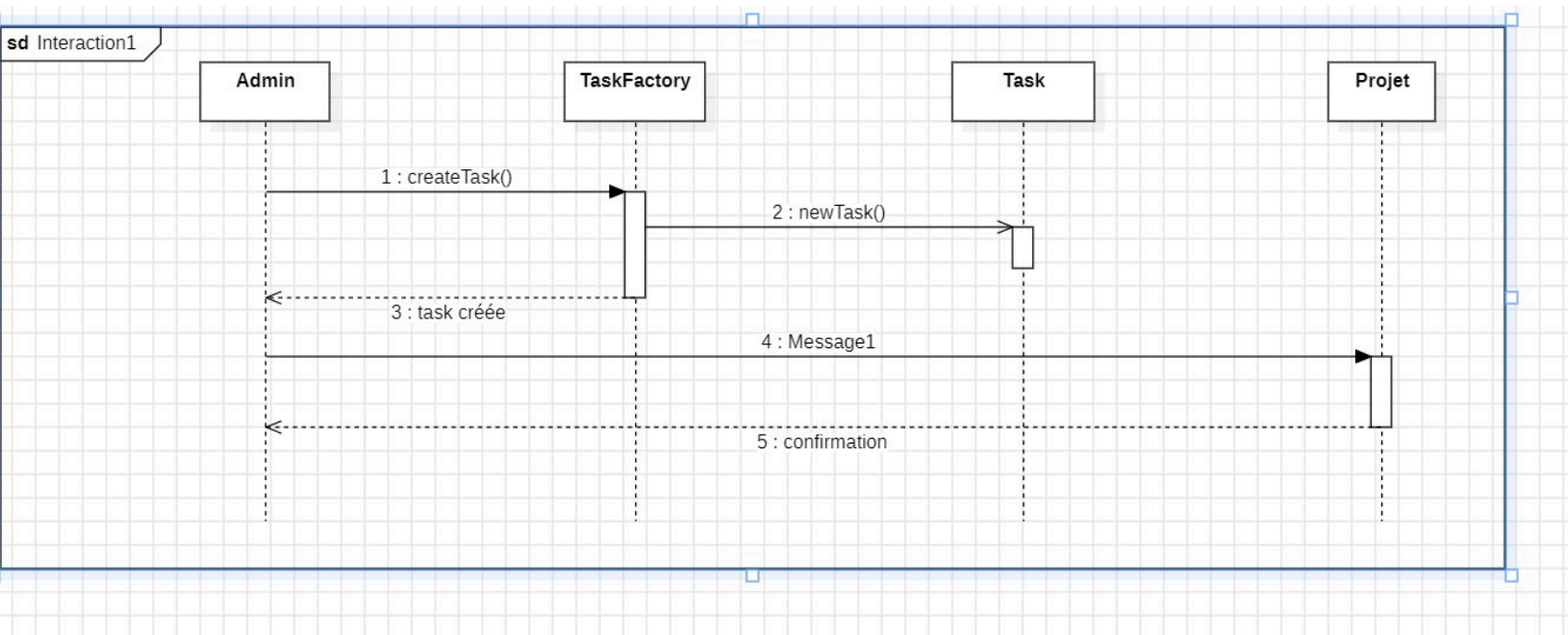
Observer

Le pattern Observer a été utilisé pour permettre la notification automatique des utilisateurs lorsqu'une tâche leur est attribuée.

Factory

Le pattern Factory a été utilisé dans la classe TaskFactory afin de centraliser et simplifier la création des objets Task.

Diagramme de séquence: Création d'une tâche



Le diagramme de séquence « Création d'une tâche » représente le déroulement chronologique des interactions entre les différents objets / les périodes d'activités / les messages synchrones et Asynchrones ... du système lors de la création d'une nouvelle tâche.

Le scénario commence lorsque l'administrateur demande la création d'une tâche. Cette demande est envoyée à la classe TaskFactory, responsable de la création des objets Task.

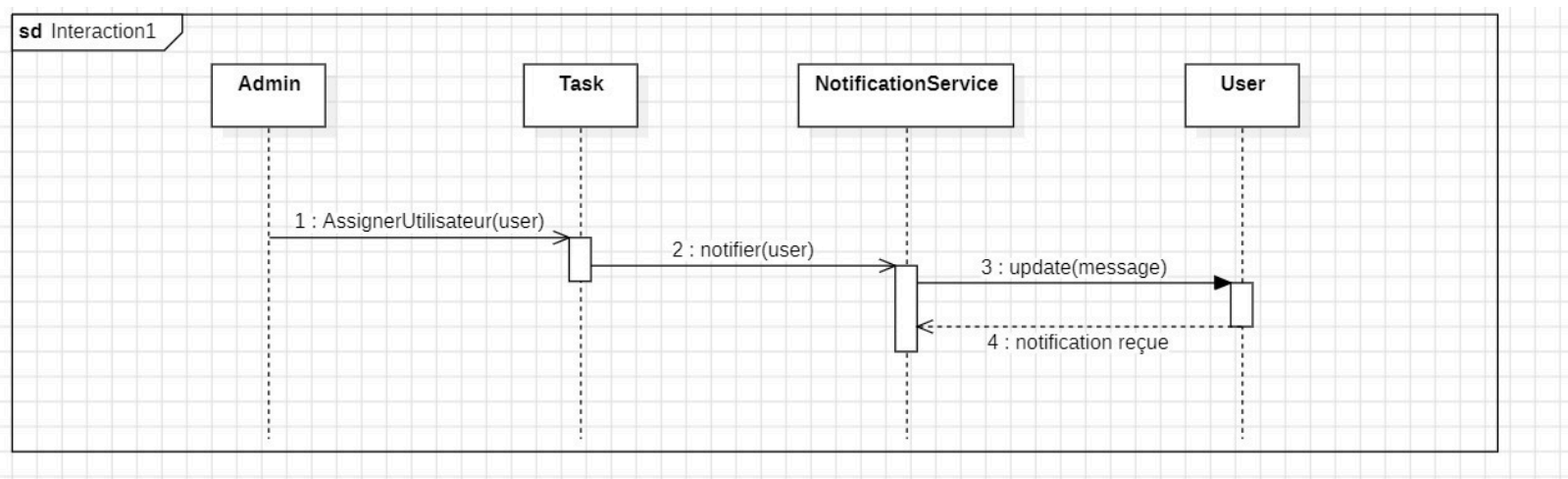
La factory crée ensuite une nouvelle instance de la tâche puis retourne l'objet créé à l'administrateur.

Après la création de la tâche, celle-ci est ajoutée au projet grâce à la méthode ajouterTache() de la classe Project.

Enfin, le système retourne une confirmation indiquant que la tâche a été correctement ajoutée au projet.

Ce diagramme met principalement en évidence l'utilisation du design pattern Factory ainsi que l'ordre des interactions entre les objets.

Diagramme de séquence: Notification d'un utilisateur



Le diagramme de séquence « Notification d'un utilisateur » illustre le processus de notification automatique lorsqu'une tâche est attribuée à un membre.

Le scénario débute lorsque l'administrateur attribue une tâche à un utilisateur à travers la méthode assignerUtilisateur().

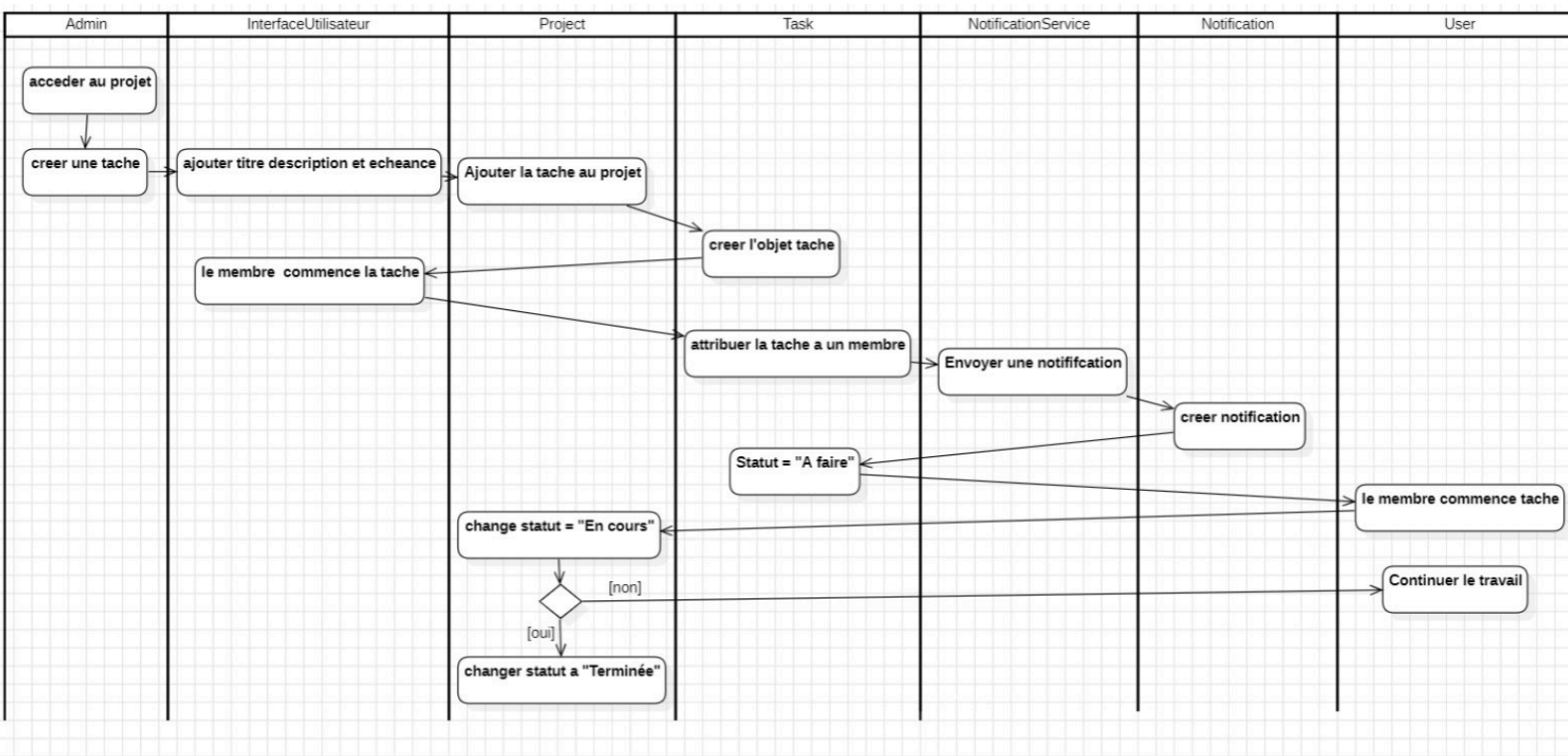
Une fois la tâche attribuée, le système appelle le service NotificationService afin d'envoyer une notification à l'utilisateur concerné.

Le service de notification transmet ensuite le message à l'utilisateur via la méthode update().

Enfin, l'utilisateur reçoit la notification confirmant qu'une nouvelle tâche lui a été assignée.

Ce diagramme met en avant l'utilisation combinée des design patterns Observer et Singleton afin d'assurer une communication efficace entre les composants du système.

Diagramme d'Activité



Le diagramme d'activité représente le cycle de vie d'une tâche dans l'application TaskFlow.

Le processus commence par la création de la tâche par l'administrateur. Une fois créée, la tâche possède le statut « À faire ».

Lorsque l'utilisateur commence à travailler sur cette tâche, le statut devient « En cours ».

Après la réalisation complète du travail demandé, la tâche passe finalement au statut « Terminée ».

Le diagramme montre également les différentes transitions possibles entre les états ainsi que l'enchaînement logique des activités.

Ce diagramme permet de mieux comprendre le comportement dynamique du système et les différentes étapes suivies par une tâche depuis sa création jusqu'à son achèvement.

Conclusion :

À travers ce mini-projet, nous avons appliqué plusieurs concepts importants de la conception logicielle tels que la modélisation UML, les relations entre classes et l'utilisation des design patterns.

Les différents diagrammes (use case, classe , séquence, activité) réalisés ont permis de représenter aussi bien la structure que le comportement du système. L'intégration des design patterns Singleton, Observer et Factory a également contribué à améliorer l'organisation, la flexibilité et la maintenabilité de l'application.

Ce projet nous a permis de mieux comprendre les étapes nécessaires à la conception d'une application logicielle collaborative et de mettre en pratique les principes étudiés dans le module de conception logicielle.